Linguaggi Formali e Compilatori
(Formal Languages and Compilers)

prof. S. Crespi Reghizzi, prof. Angelo Morzenti
(prof. Luca Breveglieri)

Prova scritta - 25 settembre 2013 - Parte I: Teoria

CON SOLUZIONI - A SCOPO DIDATTICO LE SOLUZIONI SONO MOLTO ESTESE E COMMENTATE VARIAMENTE - NON SI RICHIEDE CHE IL CANDIDATO SVOLGA IL COMPITO IN MODO ALTRETTANTO AMPIO, BENSÌ CHE RISPONDA IN MODO APPROPRIATO E A SUO GIUDIZIO RAGIONEVOLE

AVVISO - GLI ESERCIZI DI ANALISI SINTATTICA VANNO AFFRONTATI SECONDO LA TEORIA *ELR* / *ELL* - SI VEDA IL SITO WEB DEL CORSO PER MATERIALE DIDATTICO E INFORMAZIONI

NOME: _____

MATRICOLA: _____ FIRMA: _____

ISTRUZIONI - LEGGERE CON ATTENZIONE:

- L'esame si compone di due parti:
  - I (80%) Teoria:
    1. espressioni regolari e automi finiti
    2. grammatiche libere e automi a pila
    3. analisi sintattica e parsificatori
    4. traduzione sintattica e analisi semantica
  - II (20%) Esercitazioni Flex e Bison
- Per superare l'esame l'allievo deve sostenere con successo entrambe le parti (I e II), in un solo appello oppure in appelli diversi, ma entro un anno.
- Per superare la parte I (teoria) occorre dimostrare di possedere conoscenza sufficiente di tutte le quattro sezioni (1-4), rispondendo alle domande obbligatorie.
- È permesso consultare libri e appunti personali.
- Per scrivere si utilizzi lo spazio libero e se occorre anche il tergo del foglio; è vietato allegare nuovi fogli o sostituirne di esistenti.
- Tempo: Parte I (teoria): 2h.15m - Parte II (esercitazioni): 60m

1 Espressioni regolari e automi finiti 20%

1. Si consideri la grammatica G seguente, senza derivazioni auto-inclusive (assioma A):

$$G \begin{cases} A & \rightarrow & a\,B\,b\,A \mid \varepsilon \\ B & \rightarrow & c\,d\,B \mid \varepsilon \end{cases}$$

   Si risponda alle domande seguenti:

   (a) Si ricavi un'espressione regolare R per il linguaggio $L(G)$, preferibilmente considerando le regole della grammatica G come equazioni insiemistiche sui linguaggi associati ai nonterminali, e procedendo per sostituzione.

   (b) (facoltativa) Si dica se il linguaggio $L(G)$ è locale, giustificando la risposta.

   (c) Utilizzando il procedimento sistematico più semplice atto allo scopo, si ricavi un automa deterministico a stati finiti A che riconosca il linguaggio $L(G)$.

   (d) Se necessario, si minimizzi l'automa A ottenuto al punto precedente.

Solution

(a) By means of simple substitutions of nonterminals, we obtain this regular expression: $R = (\, a\, (\, c\, d\,)^*\, b\,)^*$.

(b) The language $L\,(G)$ is local because the regular expression R that generates it, which was found in the pervious point, is linear, i.e., each generator occurs in R only once, and linearity is a sufficient condition for the language to be local.

(c) We can draw the automaton A by observing that the language of G is local, as we have concluded in the previous point. The sets of initials, finals and digrams that characterize this local language are the following: $Ini = \{a\}$, $Fin = \{b\}$ and $Dig = \{ac, ab, cd, dc, db, ba\}$. The language also contains the null string ε. Notice that the local automaton A obtained in this way, is deterministic by construction.

(d) The two states q_0 and q_b are indistinguishable, as well as the states q_a and q_d. Therefore the minimal automaton has three states, as shown in the figure.

All the steps are illustrated in the figure below.

$$\begin{cases} A \to aBbA \mid \varepsilon \\ B \to cdB \mid \varepsilon \end{cases}$$

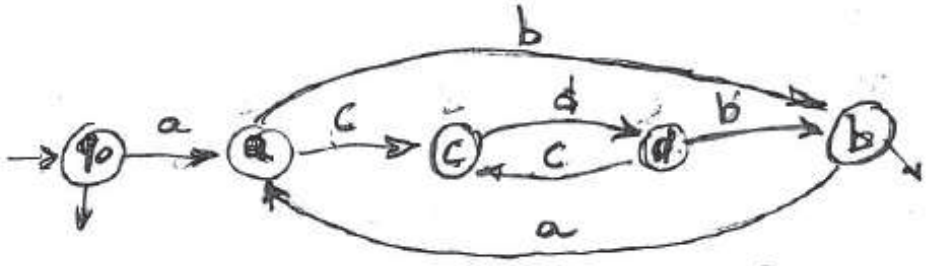
$$B = (cd)^*$$

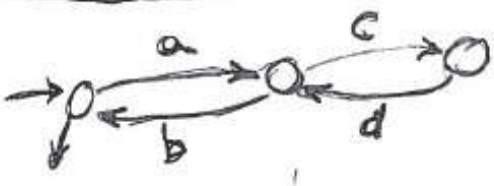
$$A \to a(cd)^* bA \mid \varepsilon$$

$$R = (a(cd)^* b)^*$$

$Ini(R) = \{a\}$

$Dig(R) = \{ac, ab, cd, dc, db, ba\}$ $Fin(R) = \{b\}$





q_a				
q_c				
q_d		$(q_c\, q_c)$ $(q_b\, q_b)$		
q_b	$(q_a\, q_a)$			
	q_0	q_a	q_c	q_b

2 Grammatiche libere e automi a pila 20%

1. Si consideri la grammatica G seguente (assioma S), di tipo *BNF* e non ambigua, con alfabeto terminale $\Sigma = \{ a, b, c \}$:

$$G\colon S \to a\ S\ a \ \mid\ b\ S\ b \ \mid\ c$$

La grammatica G genera il linguaggio delle palindomi sulle lettere a e b, con la lettera c per centro.

Si risponda alle domande seguenti:

(a) Si scriva una grammatica G', di tipo *BNF* e non ambigua, che genera il linguaggio differenza $L(G) - \{ a\,c\,a \}$ ossia il linguaggio delle palindromi senza la singola palindrome $a\,c\,a$.

(b) Si scriva una grammatica G'', di tipo *BNF* e non ambigua, che genera il linguaggio differenza $L(G) - \{ a^n\ c\ a^n \mid \ n \geq 1 \}$ ossia il linguaggio delle palindromi senza le infinite palindromi $a^n\,c\,a^n$ per tutti gli $n \geq 1$.

(c) (facoltativa) Si scriva una grammatica G''', di tipo *BNF* e non ambigua, che genera il linguaggio differenza $L(G) - \Sigma^*\,b\,a\,\Sigma^*$ ossia il linguaggio delle palindromi senza tutte quelle dove una lettera a viene subito dopo una lettera b.

Solution

(a) Here is a possible grammar G' (axiom S), of the required type:

$$G' \begin{cases} S & \to & a\,T\,a \mid b\,T\,b \mid b\,c\,b \mid c \\ T & \to & a\,T\,a \mid b\,T\,b \mid a\,c\,a \mid b\,c\,b \end{cases}$$

Grammar G' first generates the palindromes of length ≤ 3 through S (and skips palindrome $a\,c\,a$) and then generates the palindromes of length > 3 through T.

(b) Here is a possible grammar G'' (axiom S), of the required type:

$$G'' \begin{cases} S & \to & X \mid c \\ X & \to & a\,X\,a \mid b\,Y\,b \\ Y & \to & a\,Y\,a \mid b\,Y\,b \mid c \end{cases}$$

Clearly it is required to generate all and only the palindromes that contain one or more letters b, plus the shortest string c. Grammar G'' does so by forcing every derivation to pass through rule $X \to b\,Y\,b$, which generates the first letter b (others may follow through rule $Y \to b\,Y\,b$), except the derivation of string c through the direct axiomatic rule $S \to c$.

Notice that rule $X \to b\ X\ b$ is missing, and for a good reason. If it were present, then grammar G'' would be ambiguous, which we must avoid. In fact a grammar with such a rule could generate a few letters b before the one forced by rule $X \to b\,Y\,b$, or could simply defer them after passing through $X \to b\,Y\,b$ (which must happen for terminating). For instance, with the additional rule $X \to b\,X\,b$ even the simple palindrome $b\,b\,c\,b\,b$ would become ambiguous.

An alternative formulation of grammar G'', also non-ambiguous, is the following (axiom S):

$$G'' \begin{cases} S & \to & X \mid c \\ X & \to & a\,X\,a \mid b\,X\,b \mid b\,Y\,b \\ Y & \to & a\,Y\,a \mid c \end{cases}$$

where the asymmetry is moved onto nonterminal Y.

(c) Here is a possible grammar G''' (axiom S), of the required type:

$$G''' \begin{cases} S & \to & A \mid B \\ A & \to & a\,A\,a \mid c \\ B & \to & b\,B\,b \mid b\,c\,b \end{cases}$$

Clearly if we want to exclude the adjacence $b\,a$, then we automatically exclude also the reflected adjacence $a\,b$. Therefore we have to exclude all the palindromes that contain both letters a and b, as if both letters were present in one string,

then either one should come first. So we are left with the union of the two sublanguages of the palindromes of letters a and b, respectively. But we must avoid ambiguity, so we have to (arbitrarily) allow either A or B, but not both, or directly the axiom S, to generate the shortest string c. This is what grammar G''' is designed to do, e.g., with the task of generating c in charge of A.

2. Nel linguaggio *PL/I* ci sono vari tipi di costrutti iterativi, descritti qui sotto con esempi. Questi esempi contengono anche istruzioni diverse da quella iterativa, ed espressioni di vario tipo, che però non sono oggetto di questa domanda.

simple DO statement (the statements in the DO-body are executed one time only) Ex.:

```
DO;
   PUT LIST ('More data');
   GET LIST (VALUE);
   C = C + VALUE;
END;
```

DO REPEAT statement Example:

```
DECLARE (HEAD, P, NEXT) POINTER;
...
DO P = HEAD REPEAT (P->NODE.NEXT)
   WHILE (P^ = NULL);
   ...
END;
```

DO WHILE and DO UNTIL statements Examples:

```
DO WHILE (EOF = 0);
   READ FILE(F) INTO(REC);
END;

DO UNTIL (B = 1);
   ARRAY(B) = B;
   B = B - 1;
END;

DO WHILE (EOF = 0)
   UNTIL (ALPHA = 'end');
   READ FILE(F) INTO(REC);
END;
```

iterative DO statement Ex.s:

```
DO K = 1 TO 10;
   ...
END;

DO K = 10 TO 1 BY -1;
   ...
END;

DO K = 1 TO 10 WHILE (B < 0);
   ...
END;

DO A(J) = B + 1 TO N BY -M
   WHILE (T);
   ...
END;
```

DO list statement In this form none of the TO, BY or REPEAT options are used, but multiple instances of the start expression are specified. The body is executed once for each specified start value. Example:

```
DO name 'Bob', 'Ann', 'Fred', 'Joan';
   READ FILE(F) KEY(name) INTO(REC);
   CALL SEND_REPT;
END;
```

complex iterative DO statement It can include multiple instances of the iteration-spec. The DO list statement described above is a simple example of this. Example:

```
DO J = 1 TO 9
   WHILE (B(J)^ -= 'xx'),
   J = 16 TO 20 BY 2;
   READ FILE(F) INTO(REC);
   B(J) = SUBSTR(REC,6,2);
END;
```

Alcune precisazioni: nelle clausole di controllo dei `DO`, le *espressioni* e le *variabili* vanno considerate come simboli terminali `exp` e `var`, rispettivamente; il corpo dei `DO` contiene una o più istruzioni, anche del tipo `DO` stesso; ogni altra istruzione sarà denotata genericamente dal terminale `stat`; e le dichiarazioni di variabile vanno trascurate.

Si scriva una grammatica G, di tipo *EBNF* e non ambigua, per generare tali costrutti `DO`. È preferibile che la struttura della grammatica G rispecchi la classificazione precedente dei costrutti `DO`.

Solution

Here is a possible solution (axiom DO_STAT), strictly derived from the given examples:

$$
\begin{array}{rcl}
\langle \mathsf{DO_STAT} \rangle & \rightarrow & \mathsf{DO}\ \langle \mathsf{DO_CLAUSE} \rangle\ \text{`;'}\ \langle \mathsf{STAT_LIST} \rangle\ \mathsf{END} \\
\hline
\langle \mathsf{DO_CLAUSE} \rangle & \rightarrow & \varepsilon \mid \langle \mathsf{W_CLAUSE} \rangle \mid \langle \mathsf{U_CLAUSE} \rangle \\
& & \mid \langle \mathsf{W_CLAUSE} \rangle\ \langle \mathsf{U_CLAUSE} \rangle \\
& & \mid \mathsf{var}\ \text{`='}\ \mathsf{exp}\ \mathsf{REPEAT}\ \text{`('}\ \mathsf{stat}\ \text{`)'}\ \langle \mathsf{W_CLAUSE} \rangle \\
& & \mid \langle \mathsf{ITER_CLAUSE} \rangle\ \left(\text{`,'}\ \langle \mathsf{ITER_CLAUSE} \rangle \right)^* \\
& & \mid \mathsf{var}\ \mathsf{exp}\ \left(\text{`,'}\ \mathsf{exp} \right)^* \\
\langle \mathsf{STAT_LIST} \rangle & \rightarrow & \left(\left(\langle \mathsf{DO_STAT} \rangle \mid \mathsf{stat} \right) \text{`;'} \right)^* \\
\hline
\langle \mathsf{W_CLAUSE} \rangle & \rightarrow & \mathsf{WHILE}\ \text{`('}\ \mathsf{exp}\ \text{`)'} \\
\langle \mathsf{U_CLAUSE} \rangle & \rightarrow & \mathsf{UNTIL}\ \text{`('}\ \mathsf{exp}\ \text{`)'} \\
\langle \mathsf{ITER_CLAUSE} \rangle & \rightarrow & \mathsf{var}\ \text{`='}\ \mathsf{exp}\ \mathsf{TO}\ \mathsf{exp}\ \left[\mathsf{BY}\ \mathsf{exp} \right]\ \left[\langle \mathsf{W_CLAUSE} \rangle \right]
\end{array}
$$

We consider only the DO statement, and we drop any other statement type as well as all the declarations. This solution is reasonably extracted from the given examples, without trying to generalize. It is extended and non-ambiguous. It is detailed at the level of statement and expression, with no further expansion.

The explanations below, also taken from the *PL/I* manual, give a more comprehensive overview with a few possible generalizations.

We quote from *Open PL/I Language Reference Manual - Micro Focus Documentation*:

DO statement:

Purpose: begins a sequence of statements to be executed as a group, possibly to be executed iteratively.

Syntax:

```
DO [while-until-spec]| index = iteration-spec [, iteration-spec] ...];
```

where while-until-spec is:

```
WHILE(test-expression)[UNTIL(test-expression)]
```

or

```
UNTIL(test-expression)[WHILE(test-expression)]
```

and where iteration-spec is:

```
start[to-by-spec|repeat-spec][while-until-spec]
```

where to-by-spec is:

`TO finish[BY increment]`

or

`BY increment[TO finish]`

and where repeat-spec is:

`REPEAT next`

Parameters:

`test-expression`

Any expression that yields a scalar bit string value of length ≥ 1. If any bit of the value is 1, the test-expression is true; otherwise, it is false. Test-expression must be enclosed in parentheses.

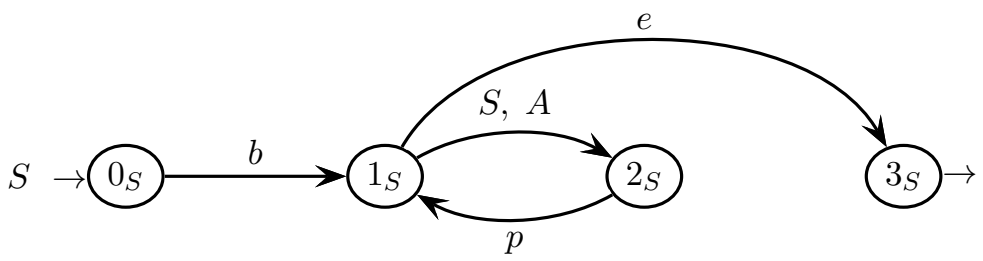
The reader may wish to reconsider the proposed solution and adjust it according to the above clarifications.

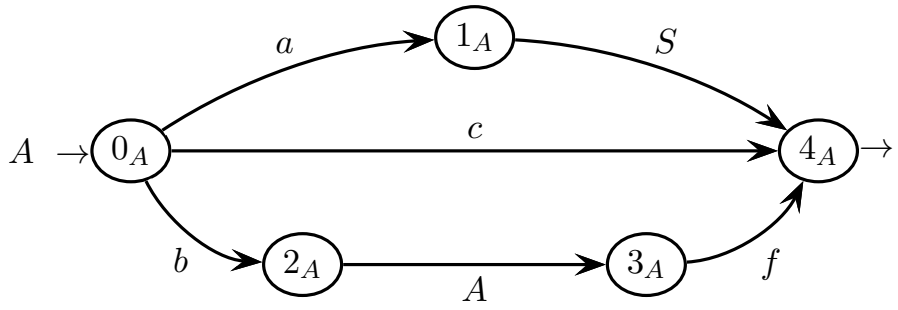
3 Analisi sintattica e parsificatori 20%

1. Si consideri la grammatica G seguente (assioma S), di tipo *EBNF*:

$$G \begin{cases} S \rightarrow b\,\big(\,(\,S \mid A\,)\,p\,\big)^*\,e \\ A \rightarrow a\,S \mid c \mid b\,A\,f \end{cases}$$

La grammatica G è rappresentata come rete di macchine ricorsive, come segue:





Si risponda alle domande seguenti:

(a) Si costruisca la porzione di automa pilota sufficiente a mostrare che la grammatica G non è di tipo $ELL(1)$.

(b) Si consideri la stringa $b\,b\,c\,f\,p\,e \in L(G)$ e se ne disegni l'albero sintattico.

(c) Si costruisca la porzione di automa pilota sufficiente per l'analisi ascendente della stringa $b\,b\,c\,f\,p\,e \in L(G)$, e si verifichi che tale porzione del pilota soddisfa la condizione $ELR(1)$.

(d) (facoltativa) Si tratteggi la simulazione dell'analisi di detta stringa, mostrando le operazioni di spostamento e riduzione, e la costruzione dell'albero sintattico.

ATTENZIONE: si veda l'avviso sul frontespizio del testo.

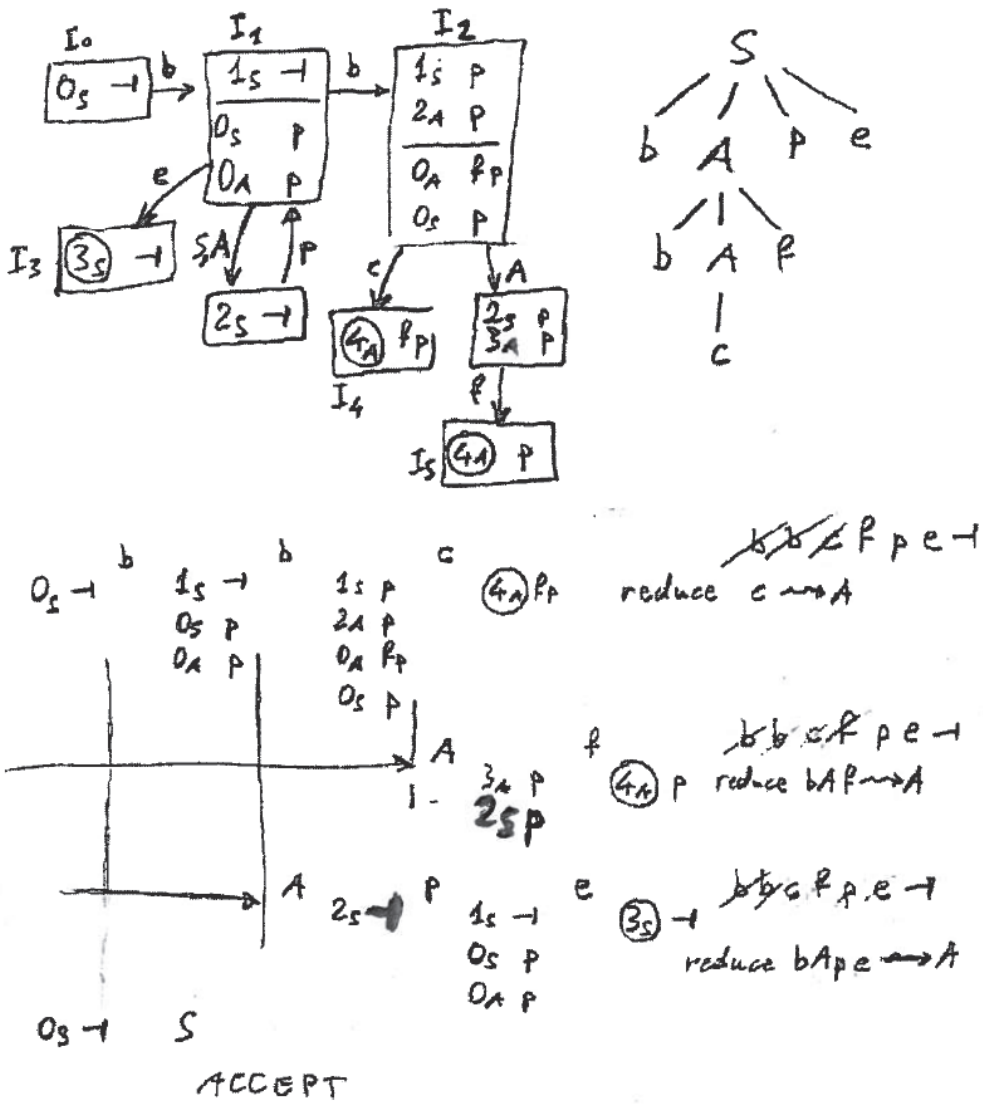
Solution

(a) The portion of the pilot showing that the grammar is not $ELL(1)$ consists of m-states I_0, I_1, and I_2 in the figure below: the Single Transition Property is not satisfied.

(b) The syntax tree for string $b\,b\,c\,f\,p\,e \in L\,(G)$ is in the picture below.

(c) The picture reports the relevant fragment of the pilot: there is no convergent edge and the m-states I_3, I_4, and I_5 do not have any RR nor SR conflict.

(d) The trace of the analysis is reported in the bottom part of the figure.



4 Traduzione e analisi semantica 20%

1. Si considerino le espressioni booleane (in forma infissa) con operatori binari *or* "$\vee$" ed *and* "$\wedge$", variabili dirette e negate rispettivamente rappresentate con a e b, e parentesi "(" e ")" con qualunque livello di annidamento. Ecco la grammatica G di queste espressioni (assioma E), di tipo *BNF* e non ambigua:

$$G \begin{cases} E & \rightarrow & E \ \vee \ T \ | \ T \\ T & \rightarrow & T \ \wedge \ F \ | \ F \\ F & \rightarrow & a \ | \ b \ | \ (\ E \ ) \end{cases}$$

Le precedenze tra operatori sono: *and* precede *or*. Ecco tre espressioni di esempio:

$$a \ \vee \ b \ \wedge \ a \qquad\qquad a \ \vee \ (\ a \ \wedge \ a \ ) \qquad\qquad a \ \wedge \ (\ a \ \vee \ b \ )$$

Nella seconda espressione le parentesi non sono necessarie, ma neppure vietate.

La forma duale di un'espressione booleana è definita scambiando *or* con *and*, e scambiando le variabili dirette con quelle negate. Le parentesi presenti nell'espressione sorgente, rimangono nella forma duale. Per esempio:

$$b \ \vee \ a \ \wedge \ (\ a \ \vee \ b \ ) \ \vee \ a \ \Rightarrow \ \underbrace{a \ \wedge \ \big(\ b \ \vee \ (\ b \ \wedge \ a \ ) \ \big) \ \wedge \ b}_{\text{forma duale}}$$

Si noti che vanno conservate le precedenze degli operatori contenuti nell'espressione sorgente, e che dunque può essere necessario aggiungere parentesi alla forma duale.

Si risponda alle domande seguenti:

(a) Si scriva una grammatica di traduzione G_τ (o uno schema sintattico), non estesa (*BNF*) e non ambigua, che calcola la forma duale delle espressioni booleane, e si disegni l'albero di traduzione dell'esempio dato sopra.

(b) (facoltativa) Si considerino espressioni booleane sorgente definite come sopra, ma senza parentesi. Si scriva un'espressione regolare di traduzione R_τ, non ambigua, che calcola la forma duale delle espressioni booleane così ristrette. Può accadere che l'espressione tradotta contenga parentesi per conservare le precedenze.

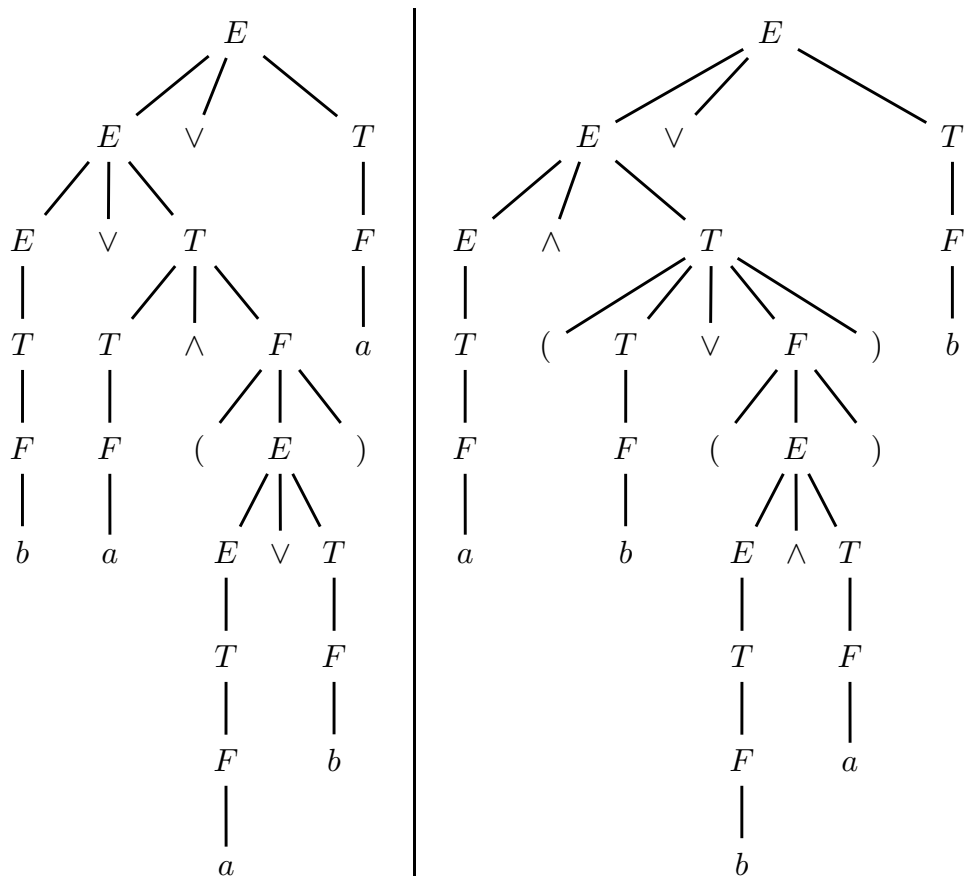
Solution

(a) Here is a possible translation scheme G_τ (axiom E), of type *BNF*, non-ambiguous and left-associative, which uses grammar G as the source one:

$$G_\tau \begin{cases} E \to E \vee T \mid T & E \to E \wedge T \mid T \\ T \to T \wedge F \mid F & T \to (\, T \vee F \,) \mid F \\ F \to a \mid b \mid (\, E \,) & F \to b \mid a \mid (\, E \,) \end{cases}$$

The source grammar is simply the well known *BNF* non-ambiguous grammar G of the arithmetic expressions, but here it is tuned to the logical operators with the required precedences. The destination grammar is structurally identical to the source one, as it must be, turns each operator to the dual one, and deploys a parenthesis nest around each *or* operator that is the dual of an *and* operator, besides complementing the variables. The parentheses already present in the source logical expression, are left unchanged.

Here are the syntax tree of the source string $b \vee a \wedge (\, a \vee b \,) \vee a$ and that of the translated string $a \wedge \big(\, b \vee (\, b \wedge a \,) \,\big) \wedge b$:



Notice that parentheses are deployed around the *and* operator that is translated into an *or* operator, to keep the higher precedence of the original *and*. The original parentheses present in the source logical expressions, are just preserved.

In the case the source logical expression contains a long chain of *and*'s, the chain is translated into an as long chain of *or*'s with nested parentheses laid at each step in a left-associative way. For instance:

$$a \ \wedge \ b \ \wedge \ a \ \vee \ b \ \Rightarrow \ \big((b \ \vee \ a) \ \vee \ b \big) \ \wedge \ a$$

This is a simple solution and is definitely acceptable, though all such parentheses except the outermost nest, are unnecessary. But we could refine the scheme with more nonterminals and rules, and thus have a translated chain with only one parenthesis nest deployed around. Here is a possible scheme G'_τ (axiom E):

$$G'_\tau \left\{ \begin{array}{lcl} E & \rightarrow & E \ \vee \ T \ | \ T \\ T & \rightarrow & T' \ \wedge \ F \ | \ F \\ T' & \rightarrow & T' \ \wedge \ F \ | \ F \\ F & \rightarrow & a \ | \ b \ | \ (\ E \ ) \end{array} \right. \qquad \begin{array}{lcl} E & \rightarrow & E \ \wedge \ T \ | \ T \\ T & \rightarrow & (\ T' \ \vee \ F \ ) \ | \ F \\ T' & \rightarrow & T' \ \vee \ F \ | \ F \\ F & \rightarrow & b \ | \ a \ | \ (\ E \ ) \end{array}$$

A chain of *and*'s must start by the source rule $T \rightarrow T' \ \wedge \ F$, and the destination rule $T \rightarrow (\ T' \ \vee \ F \ )$ will turn the *and* into an *or* and deploy a parenthesis nest around. More *and*'s can be chained by nonterminal T', yet no more parentheses will be deployed in the translation. So we get a translated logical expression like:

$$a \ \wedge \ b \ \wedge \ a \ \vee \ b \ \Rightarrow \ \big(b \ \vee \ a \ \vee \ b \big) \ \wedge \ a$$

which is as valid as that before. The reader may check the correctness by drawing the source and destination syntax trees. See also question (b).

(b) To proceed stepwise, we first give a possible regular source expression. Call V (it stands for *variables*) the subexpression "$(\ a \ | \ b \ )$", and in the following imagine to replace this symbol V by such a subexpression, including the brackets around. Then here is a straightforward regular source expression R:

$$R = V \ (\ \wedge \ V \ )^* \ \Big(\ \vee \ \big(V \ (\ \wedge \ V \ )^* \big) \ \Big)^*$$

The *and* operator is crucial because in the translated string (where it appears as an *or* operator) it needs parentheses around, to keep precedence. Thus we may wish to distinguish the terms that consist of only one variable, from those that consist of one or more *and*'s. Here it is a reformulation of R, called R':

$$R' = \Big(V \ | \ V \ (\ \wedge \ V \ )^+ \Big) \ \Big(\ \vee \ \Big(V \ | \ V \ (\ \wedge \ V \ )^+ \Big) \ \Big)^*$$

Now we can move to the translation. Call $\overline{V}$ the regular translation subexpression "$(\ \frac{a}{b} \ | \ \frac{b}{a} \ )$", similarly to what we have done before. Then here is a possible regular translation expression R'_τ:

$$R'_\tau = \Big(\overline{V} \ | \ \tfrac{\varepsilon}{\text{`(`}} \ \overline{V} \ \big(\tfrac{\wedge}{\vee} \ \overline{V} \big)^+ \ \tfrac{\varepsilon}{\text{`)`}} \Big) \ \Big(\tfrac{\vee}{\wedge} \ \Big(\overline{V} \ | \ \tfrac{\varepsilon}{\text{`(`}} \ \overline{V} \ \big(\tfrac{\wedge}{\vee} \ \overline{V} \big)^+ \ \tfrac{\varepsilon}{\text{`)`}} \Big) \ \Big)^*$$

Do not confuse between parentheses as meta-symbols (without apices) and as terminals (with apices). In the case the source logical expression contains a long chain of *and*'s, the dual form thereof will have only one parenthesis nest around. For instance:

$$a \ \wedge \ b \ \wedge \ a \ \vee \ b \ \Rightarrow \ \big(b \ \vee \ a \ \vee \ b \big) \ \wedge \ a$$

This is the same behavior as of scheme G'_τ. But here such a behavior is a necessity: in fact a regular expression cannot generate a balanced parenthesis structure of arbitrary depth, as the context-free scheme G_τ can instead do.

A final observation for both questions (a) and (b): if at the top level the source logical expression only consists of a chain of *and* operators, then both the two translation schemes and the regular translation expression proposed deploy a parenthesis nest around the whole dual form. This outermost nest is clearly unnecessary. However it can be avoided by resorting to a technique similar to that employed for designing the scheme G'_τ and the regular translation expression R'_τ: differentiating the behavior of the axiom and distinguishing the case where there are only *and*'s, respectively.

Conceptually such a refinement does not add anything else to what we have already found. However here are two possible solutions, namely a scheme G''_τ with axiom S:

$$G''_\tau \begin{cases} S \rightarrow E \vee T \mid T' & S \rightarrow E \wedge T \mid T' \\ E \rightarrow E \vee T \mid T & E \rightarrow E \wedge T \mid T \\ T \rightarrow T' \wedge F \mid F & T \rightarrow (\, T' \vee F \,) \mid F \\ T' \rightarrow T' \wedge F \mid F & T' \rightarrow T' \vee F \mid F \\ F \rightarrow a \mid b \mid (\, E \,) & F \rightarrow b \mid a \mid (\, E \,) \end{cases}$$

and a regular translation expression R''_τ:

$$\begin{aligned} R''_\tau \;=\; & \Big(\overline{V} \mid \tfrac{\varepsilon}{\text{`('}}\, \overline{V} \, \big( \tfrac{\wedge}{\vee}\, \overline{V} \big)^+ \, \tfrac{\varepsilon}{\text{`)'}} \Big) \Big(\tfrac{\vee}{\wedge} \Big(\overline{V} \mid \tfrac{\varepsilon}{\text{`('}}\, \overline{V} \, \big( \tfrac{\wedge}{\vee}\, \overline{V} \big)^+ \, \tfrac{\varepsilon}{\text{`)'}} \Big) \Big)^+ \\ & \mid \overline{V} \, \big(\tfrac{\wedge}{\vee}\, \overline{V} \big)^* \end{aligned}$$

Clearly both the scheme G''_τ and the regular translation expression R''_τ translate the logical source expression $a \wedge b$ into the dual form $b \vee a$ without deploying any parenthesis nest around, despite the source operator is *and*.

2. Si deve ricercare quante volte compare un certo identificatore (*query*) in un insieme di record e di oggetti atomici (c'è sempre almeno un oggetto atomico). Ogni record può contenere sia altri record sia campi atomici, ed è specificato dalla sintassi seguente:

$$
\begin{array}{lcll}
L & \rightarrow & \text{id}: \text{record } L \text{ end } L & \text{- - “id” ha l'attributo destro } n \\
L & \rightarrow & \text{id}: \text{atomic } L & \text{- - idem} \\
L & \rightarrow & \text{id}: \text{atomic} & \text{- - idem}
\end{array}
$$

L'identificatore “id” è associato a un attributo destro n inizializzato con il nome.

Il nome (ossia stringa) da ricercare sta nell'attributo destro q del terminale “query” nella regola seguente:

$$
S \rightarrow L \text{ query} \qquad \text{- - “query” ha l'attributo destro } q
$$

Per esempio nei record seguenti:

A: record C: atomic end $B2$: record A: atomic end A: atomic

il nome A compare tre volte, mentre i nomi C e $B2$ compaiono una sola volta per ciascuno.

Si risponda alle domande seguenti:

(a) Si scriva una grammatica con attributi per calcolare nell'attributo *tot* il numero di comparse del nome q. Si scrivano le regole semantiche nel quadro predisposto. Se si usano altri attributi, per ognuno nel quadro si indichino tipo e finalità.

Quadro degli attributi predefiniti ed eventuali da aggiungere per la domande (a):

tipo	*nome*	*associazione*	*dominio*	*significato*
destro	q	query L	stringa	nome da cercare
destro	n	id	stringa	nome del campo o del record
sinistro	*tot*	L S	intero	numero di comparse

(b) (facoltativa) Si costruiscano almeno in parte le procedure semantiche che calcolano con una scansione (one-sweep) gli attributi. Se ciò non fosse possibile, si spieghi perché.

#	*sintassi*	*semantica* - domanda (a) - da completare
1:	$S_0 \rightarrow L_1 \ \text{query}_2$	
2:	$L_0 \rightarrow \begin{array}{l} \text{id}_1 : \ \text{record} \\ \quad L_2 \\ \text{end} \\ L_3 \end{array}$	
3:	$L_0 \rightarrow \text{id}_1 : \ \text{atomic} \ L_2$	
4:	$L_0 \rightarrow \text{id}_1 : \ \text{atomic}$	

Solution

(a) Attributes:

type	*name*	*association*	*domain*	*meaning*
right	q	query L	string	name to be searched
right	n	id	string	name of record or field
left	tot	L S	integer	number of occurrences

No need to add more attributes.
Attribute grammar:

#	*syntax*	*semantic* - question (a) - completed
1:	$S_0 \rightarrow L_1$ query$_2$	$q_1 := q_2$ $tot_0 := tot_1$

2:	$L_0 \rightarrow$ id$_1$: record
	L_2
	end
	L_3
		$q_2 := q_0$
		$q_3 := q_0$
		if $n_1 = q_0$ **then**
		$tot_0 := tot_2 + tot_3 + 1$
		else
		$tot_0 := tot_2 + tot_3$
		end if

3:	$L_0 \rightarrow$ id$_1$: atomic L_2	$q_2 := q_0$
		if $n_1 = q_0$ **then**
		$tot_0 := tot_2 + 1$
		else
		$tot_0 := tot_2$
		end if

4:	$L_0 \rightarrow$ id$_1$: atomic	**if** $n_1 = q_0$ **then**
		$tot_0 := 1$
		else
		$tot_0 := 0$
		end if

(b) It is easy to see that the attribute grammar is of type one-sweep: first the inherited attribute q is propagated top-down, and then the synthesized attribute tot is computed bottom-up depending on the propagated value of q.

We can notice that the syntax is of type $LL\,(4)$, since by a guide length equal to three tokens we can easily distinguish the alternative rules number 2 and $3, 4$ (respectively identified by tokens "record" and "atomic"), and that by a guide length equal to four tokens we can distinguish between the alternative rules 3 and 4 (respectively identified by "id" and "end" or "query"). So we can build the syntax tree by means of a recursive descent syntactic analyzer. However the semantic analysis simply relies on a syntax tree, no matter how it is obtained.

Here is the meaningful semantic procedure, for the nodes of nonterminal L:

```
procedure L ( in q; out tot )
    var
        n1, q2, q3 : string
        tot2, tot3 : integer
    case node type of
    2 : begin
            n1 := "id"
            q2 := q
            q3 := q
            call L ( q2, tot2 )
            call L ( q3, tot3 )
            if n1 = q0 then
                tot := tot2 + tot3 + 1
            else
                tot := tot2 + tot3
            end if
        end
    3 : begin
            n1 := "id"
            q2 := q
            call L ( q2, tot2 )
            if n1 = q0 then
                tot := tot2 + 1
            else
                tot := tot2
            end if
        end
    4 : begin
            n1 := "id"
            if n1 = q then
                tot := 1
            else
                tot := 0
            end if
        end
    end case
end procedure
```

Attribute n is simply calculated from the token. It would be possible to reduce the number of local variables as well as simplify the conditionals, but this is just program optimization.

Here is the axiomatic procedure:

```
procedure S ( out tot )
    var
        q1, q2: string
        tot1: integer
    begin
        q2 := "query"
        q1 := q2
        call L ( q1, tot1 )
        tot := tot1
    end
end procedure
```

Attribute q is simply calculated from the token. Also here optimizations are possible, for instance eliminating variable q_2.

Finally, here is the main program:

```
program SEMANTIC_ANALIZER
    var
        tot: integer
    begin
        - - build the syntax tree
        call S ( tot )
        print ("The number is %d.", tot)
    end
end program
```

We can soon notice that the recursive descent syntactic analyzer could be integrated with the semantic one, but for the assignment $q_1 := q_2$ in rule 1, which is right-to-left and not left-to-right ! However we could reshape the rule in this way: $S \rightarrow$ query L, for such a rule is just meant as a way to input the query to the syntax.

Of course it is possible to send the query value directly to the semantic analyzer, rather than incorporating it in the syntax. This is once again just program optimization.